TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
KALANKI, KATHMANDU



A Minor Project Final Report On

"Crop Disease Detection Using Convolutional Neural Network"
[ENCT 354]

Submitted by:

Aaditya Kumar Sah (ACE080BCT001)
Ayush Kumar Sah (ACE080BCT017)
Gautam Jayswal (ACE080BCT029)

Submitted to:

Department of Electronics and Computer Engineering
Advanced College of Engineering and Management

9th September, 2026

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
KALANKI, KATHMANDU



A Minor Project Final Report On

"Crop Disease Detection Using Convolutional Neural Network"
[ENCT 354]

Submitted by:

Aaditya Kumar Sah (ACE080BCT001)
Ayush Kumar Sah (ACE080BCT017)
Gautam Jayswal (ACE080BCT029)

Submitted to:

Department of Electronics and Computer Engineering
Advanced College of Engineering and Management

Supervised by:

Er. Diwakar Dhungana

A Minor Project Report submitted to the Department of Electronics and Computer Engineering in the partial fulfillment of the requirements for degree of Bachelor of Engineering in Computer Engineering
Kathmandu, Nepal

9th September, 2026

DECLARATION

We hereby declare that the project report entitled “Crop Disease Detection Using Convolutional Neural Network”, submitted to the Department of Electronics and Computer Engineering, Advanced College of Engineering and Management, in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering, is an original work carried out by us.

We further declare that this work has not been submitted to any other university or institution for the award of any degree. All the sources used in this report have been properly acknowledged and listed in the references. We take full responsibility for the content and originality of this work.

Aaditya Kumar Sah (ACE080BCT001) _____

Ayush Kumar Sah (ACE080BCT017) _____

Gautam Jayswal (ACE080BCT029) _____

Date: September 9, 2026

TRIBHUVAN UNIVERSIY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

APPROVAL LETTER

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"Crop Disease Detection Using Convolutional Neural Network"** submitted by **Aaditya Kumar Sah** , **Ayush Kumar Sah**, and **Gautam Jayswal** in partial fulfillment of the requirements for the Bachelor's degree in Computer Engineering.

.....
Er. Diwakar Dhungana
Supervisor
Department of Electronics and Computer Engineering,
Advanced College of Engineering and Management

.....
Er. Roshani Ghimire
Head of Department
Department of Electronics and Computer Engineering,
Advanced College of Engineering and Management

.....

Internal Examiner
Department of Electronics and Computer Engineering,
Advanced College of Engineering and Management

9th September, 2026

COPYRIGHT

The author has agreed that the library, Advanced College of Engineering and Management, may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project was done. It is understood that recognition will be given to the report's author and the Department of Electronics and Computer Engineering, Advanced College of Engineering and Management for any use of the material of this project report. Copying publication or the other use of this project for financial gain without the approval of the Department an author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in should be addressed to:

Head of Department
Department of Electronics and Computer Engineering
Advanced College of Engineering and Management
Kalanki, Kathmandu
Nepal

ACKNOWLEDGEMENTS

We take this opportunity to express our deepest and sincere gratitude to our Project Supervisor **Er. Diwakar Dhungana**, Department of Electronics and Computer Engineering, Institute of Engineering, Advanced College of Engineering and Management, for his insightful advice, motivating suggestions, invaluable guidance, help, and support in the successful completion of this project, and also for his constant encouragement and advice throughout our Bachelor's program.

We express our deep gratitude to **Er. Roshani Ghimire**, Head of the Department of Electronics and Computer Engineering, **Er. Amit Kumar Rauniyar**, Deputy Head, Department Of Electronics and Computer Engineering, and Department of Electronics and Computer Engineering for their regular support, cooperation, and coordination.

The in-time facilities provided by the department throughout the Bachelor's program are also equally acknowledgeable.

We would like to convey our thanks to the teaching and non-teaching staff of the Department of Electronics and Communication and Computer Engineering, Advanced College of Engineering and Management for their invaluable help and support throughout the period of Bachelor's Degree. We are also grateful to all our classmates for their help, encouragement, and invaluable suggestions.

Finally, yet more importantly, I would like to express our deep appreciation to our grandparents, parents, and siblings for their perpetual support and encouragement throughout the Bachelor's degree period.

Aaditya Kumar Sah (ACE080BCT001)
Ayush Kumar Sah (ACE080BCT017)
Gautam Jayswal (ACE080BCT029)

ABSTRACT

Agriculture is the backbone of Nepal's economy, employing more than 60% of the population. However, crop diseases continue to cause significant reductions in agricultural productivity each season. Conventional disease identification relies on manual inspection by agricultural experts, which is time-consuming, expensive, and often inaccessible to farmers in remote areas. To address this challenge, this project developed an automated Crop Disease Detection System using Convolutional Neural Networks (CNNs) to classify crop leaf images into multiple disease and healthy categories.

The system utilized the PlantVillage dataset, which contains more than 54,000 annotated leaf images across 38 classes, including both healthy and diseased crops. The deep learning model was developed using Python, TensorFlow/Keras, and OpenCV for image preprocessing, model development, and evaluation. A React.js-based web interface and a Django REST API backend were also developed to allow users to upload crop leaf images and receive disease predictions in real time.

The performance of the developed model was evaluated using standard classification metrics, including accuracy, precision, recall, and F1-score. The completed system demonstrated the practical application of deep learning for automated crop disease identification through a user-friendly web-based platform. The system can assist farmers and agricultural practitioners in detecting crop diseases at an early stage, enabling timely intervention and potentially reducing crop losses. Overall, the project demonstrated the potential of CNN-based image classification as an effective solution for crop disease detection and its practical application in precision agriculture, particularly in resource-constrained environments such as Nepal.

Keyword: *Crop Disease Detection, Convolutional Neural Network (CNN), Deep Learning, Django REST API, MobileNetV2, PlantVillage Dataset, React.js, Transfer Learning*

Table of Contents

Abstract ..... **vii**

List of Figures ..... **x**

List of Tables ..... **xi**

Abbreviations ..... **xii**

Chapter 1
INTRODUCTION **1**

1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Significance of Study 2
1.5 Scope and Limitations 3
1.5.1 Scopes 3
1.5.2 Limitations 3
1.6 Outline of the Report 3

Chapter 2
LITERATURE REVIEW **4**

2.1 Theoretical Background 4
2.2 Related Works 5
2.3 Research Gap Analysis 6

Chapter 3
PROPOSED METHODOLOGY **7**

3.1 Project Approach 7
3.2 System Architecture 8
3.3 CNN Model Architecture 9
3.4 System Design 10
3.5 Technology Stack 11
3.6 Data Management 11
3.7 Testing and Validation 12
3.8 Deployment Strategy 12

Chapter 4
SYSTEM DESIGN AND IMPLEMENTATION 13
4.1 Requirements Specification 13
4.2 System Overview 15
4.3 Development Environment 17
4.4 System Implementation 17
4.4.1 Data/Input Handling 17
4.4.2 Pre-processing Steps 20
4.4.3 Module Design & Integration 23
4.4.4 Interface and Visualisation 24
4.4.5 Deployment and Execution 25
4.4.6 Challenges and Solutions 27

Chapter 5
RESULTS AND DISCUSSION 28
5.1 Results 28
5.2 Discussion 30
5.3 Comparative Analysis 30

Chapter 6
CONCLUSIONS & RECOMMENDATIONS 31
6.1 Conclusions 31
6.2 Future Enhancements 32

References 33

List of Figures

3.1 Agile Development Workflow 7
3.2 System Architecture 8
3.3 CNN Model Architecture 9
3.4 System Design 10
3.5 Data Pipeline 11
4.1 Final as-built system architecture 16
4.2 Client-side file validation in ImageUploader.jsx 18
4.3 Server-side file validation in LeafImageUploadSerializer 19
4.4 Pre-processing Steps 20
4.5 Flow of the predict_disease function 21
4.6 Implementation of predict_disease() in model_utils.py 22
4.7 Disease class 22
4.8 User interface 24
5.1 User interface 28
5.2 Training and Validation Loss 29
5.3 Performance Matrix 29

List of Tables

4.1 Mapping of development iterations to implemented system components 15
4.2 Tools and Technologies Used 17
6.1 Evaluation of project objectives against outcomes 31

LIST OF ABBREVIATIONS

AI: Artificial Intelligence

API: Application Programming Interface

CNN: Convolutional Neural Network

CORS: Cross-Origin Resource Sharing

DRF: Django REST Framework

GDP: Gross Domestic Product

HTTP: Hypertext Transfer Protocol

IOE: Institute of Engineering

JSON: JavaScript Object Notation

ML: Machine Learning

OpenCV: Open Source Computer Vision Library

ReLU: Rectified Linear Unit

REST: Representational State Transfer

RGB: Red, Green, Blue

SPA: Single Page Application

UI: User Interface

URL: Uniform Resource Locator

CHAPTER 1
INTRODUCTION

Nepal is an agrarian nation where agriculture contributes approximately 27% of GDP and employs the majority of the workforce. Despite its economic significance, the agricultural sector faces persistent challenges, particularly crop diseases caused by fungi, bacteria, and viruses. These diseases can devastate entire harvests if not identified and treated promptly. The conventional approach to disease diagnosis involves physical inspection by trained plant pathologists. This method is not scalable, is subject to human error, and is impractical for smallholder farmers who lack access to specialists. The growing availability of smartphones with cameras presents an opportunity to build low-cost, accessible diagnostic tools powered by artificial intelligence. Deep learning, particularly Convolutional Neural Networks (CNNs), has demonstrated exceptional capability in visual recognition tasks. CNNs can learn hierarchical feature representations directly from raw image data, making them ideal for leaf disease classification. This project proposes to leverage CNN-based deep learning to build an end-to-end automated crop disease detection system accessible via a web browser.

1.1 Background

Agriculture is one of the major sectors of Nepal and supports a large portion of the population. Crop diseases significantly reduce agricultural productivity and can lead to economic losses for farmers. Recent advances in Artificial Intelligence and Deep Learning have enabled automated disease detection systems that can identify plant diseases from leaf images with high accuracy.

1.2 Problem Statement

The practical problem experienced by users is the inability to rapidly and accurately diagnose crop diseases without expert intervention. Existing solutions are insufficient because they often require specialized hardware, lack user- friendly interfaces, or do not cover a wide variety of crop species. There is an urgent need for an automated, accessible, and accurate application that allows farm- ers to use standard smartphone or computer images to diagnose diseases efficiently, thereby supporting timely agricultural decision-making and reducing potential crop losses.

1.3 Objectives

1.3.1 General Objective

To develop a web-based Crop Disease Detection System using Convolutional Neural Networks (CNNs) for accurate identification of crop diseases from leaf images.

1.3.2 Specific Objectives

- To collect and preprocess crop leaf images using the PlantVillage dataset.
- To design and train a Convolutional Neural Network (CNN) model for disease classification.
- To evaluate the trained model using performance metrics such as accuracy, precision, recall, and F1-score.
- To provide farmers with a simple, reliable, and accessible tool for early crop disease detection.

1.4 Significance of Study

The developed system has successfully provided an efficient, accurate, and affordable solution for crop disease detection. It enables farmers to identify diseases at an early stage, allowing for timely treatment and reduced crop losses. The project successfully demonstrated the practical application of Artificial Intelligence and Deep Learning in agriculture, promoting smart farming practices. This study benefits farmers, agricultural researchers, students, and policy makers by deliver- ing an accessible disease diagnosis tool that enhances agricultural productivity and contributes to sustainable farming. Furthermore, the completed project serves as a foundation for future research in AI-based agricultural decision support systems.

1.5 Scope and Limitations

1.5.1 Scopes

- Developed a web-based crop disease detection system using a CNN-based MobileNetV2 model.
- Trained the model using the PlantVillage dataset to classify 38 crop disease and healthy categories.
- Integrated the trained model with a React and Django-based web application for disease prediction.
- Evaluated the model using accuracy, precision, recall, and F1-score.

1.5.2 Limitations

- The system is limited to the 38 categories available in the PlantVillage dataset.
- Prediction accuracy may decrease for poor-quality images, complex backgrounds, and real-world field conditions.
- The system considers only leaf images and does not consider environmental factors such as soil, temperature, humidity, or weather.
- The predictions are supportive and do not replace professional agricultural diagnosis.
- The system lacks out-of-distribution detection any input, including non-leaf images, is forced into one of the 38 trained classes rather than being rejected, though such misclassifications typically show a low confidence score.

1.6 Outline of the Report

The remainder of this report is organized as follows: Chapter 2 reviews the theoretical background and related works in automated crop disease detection. Chapter 3 details the methodology, system architecture, and technology stack adopted for this product. Chapter 4 presents the system design and the progress of the implementation so far. Chapter 5 discusses preliminary results, and Chapter 6 outlines the remaining tasks and the timeline to completion.

CHAPTER 2
LITERATURE REVIEW

2.1 Theoretical Background

Crop disease detection is an important application of Artificial Intelligence (AI) and Deep Learning (DL) in modern agriculture. Early detection of plant diseases helps farmers reduce crop losses, improve productivity, and minimize the excessive use of pesticides. Traditional disease identification methods rely on manual inspection by agricultural experts, which can be time-consuming, expensive, and inaccessible for farmers in remote areas.

Machine Learning (ML) has significantly improved automated image classification; however, Deep Learning techniques, particularly Convolutional Neural Networks (CNNs), have achieved superior performance by automatically learning relevant features from images. CNNs eliminate the need for manual feature extraction and are capable of recognizing complex visual patterns in crop leaf images.

A typical CNN architecture consists of convolutional layers, activation functions such as ReLU, pooling layers, and fully connected layers. The convolutional layers extract important visual features, while pooling layers reduce the spatial dimensions of feature maps. The final fully connected layers perform classification based on the extracted features.

Transfer learning has become a widely adopted technique in image classification tasks. Instead of training a model from scratch, pre-trained models such as MobileNetV2, ResNet50, VGG16, and InceptionV3 can be fine-tuned using a target dataset. Transfer learning reduces training time, improves model performance, and is especially beneficial when the available dataset is limited.

The PlantVillage dataset is one of the most widely used benchmark datasets for crop disease detection. It contains more than 54,000 labeled images representing healthy and diseased crop leaves across multiple plant species. The availability of this dataset has enabled researchers to develop and evaluate high-performance disease classification models.

In this project, a CNN model with transfer learning using MobileNetV2 is proposed to classify crop leaf images into healthy and diseased categories. The trained model will be integrated with a Django REST API backend and a React.js frontend to provide real-time disease prediction through a web application.

2.2 Related Works

Several researchers have investigated the application of deep learning techniques for crop disease detection.

Mohanty, Hughes, and Salathé [1] utilized deep learning models, including AlexNet and GoogLeNet, on the PlantVillage dataset for image-based plant disease classification. Their study achieved over 99% accuracy under controlled conditions and demonstrated the feasibility of using CNNs for automated disease diagnosis. However, the study primarily focused on classification performance and did not address real-world deployment challenges.

Ferentinos [2] conducted a comparative analysis of various deep learning architectures, including VGG, AlexNet, and Inception models, for plant disease detection. The research reported high classification accuracies and highlighted the effectiveness of deep architectures for agricultural image analysis. The study also emphasized the importance of transfer learning for improving performance on limited datasets.

Hughes and Salathé [3] introduced the PlantVillage dataset, an open-access repository containing thousands of annotated plant leaf images. The dataset became a benchmark resource for developing and evaluating machine learning and deep learning models for plant disease diagnosis.

Tm et al. [4] proposed a tomato leaf disease detection system using CNNs and demonstrated the possibility of deploying disease detection applications on mobile devices. Their work showed that lightweight CNN architectures could provide real-time predictions while maintaining satisfactory accuracy.

Chollet [5] presented comprehensive methodologies for designing, training, and optimizing deep learning models using TensorFlow and Keras. The techniques discussed in this work have become fundamental for developing modern CNN-based classification systems.

These studies collectively confirm that CNN-based approaches provide high accuracy and reliability for plant disease detection and serve as the foundation for the proposed system.

2.3 Research Gap Analysis

Although significant progress has been made in automated crop disease detection, several research gaps remain.

• Most previous studies focus primarily on achieving high classification accuracy rather than developing complete end-to-end systems for practical use by farmers.

• Many models are evaluated only on controlled datasets such as PlantVillage, while real-world agricultural environments involve varying lighting conditions, complex backgrounds, and image noise.

• Several deep learning models require high computational resources, making deployment difficult on low-cost devices or limited-resource environments.

• Existing systems often lack simple and user-friendly web interfaces that enable farmers to easily upload images and obtain disease predictions.

• Many studies do not provide scalable web-based deployment using modern technologies such as Django REST Framework and React.js.

• Limited research focuses on developing practical crop disease detection systems specifically suited to the agricultural conditions and technological accessibility of developing countries such as Nepal.

The proposed project addresses these gaps by developing a web-based crop disease detection system using a CNN model with transfer learning. The system aims to provide accurate disease prediction through an intuitive user interface, making the technology more accessible and practical for farmers.

CHAPTER 3
PROPOSED METHODOLOGY

3.1 Project Approach

The project follows an iterative, Agile-inspired development approach. The system was divided into three distinct modules: Machine Learning Pipeline, Backend API, and Frontend Application. These were developed iteratively, starting with dataset preparation and model training, followed by API development, and concluding with frontend integration.

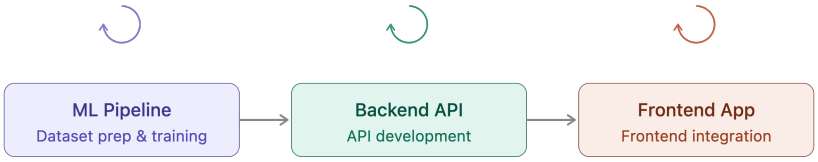


fig. 3.1: *Agile Development Workflow*

3.2 System Architecture

The system is designed as a three-tier client-server architecture. The presentation tier is a React.js single-page application (SPA) that allows users to upload leaf images and view disease predictions. The logic tier is a Django REST API that receives images, runs preprocessing and inference, and returns results. The data tier consists of the trained CNN model and the PlantVillage dataset used during training.

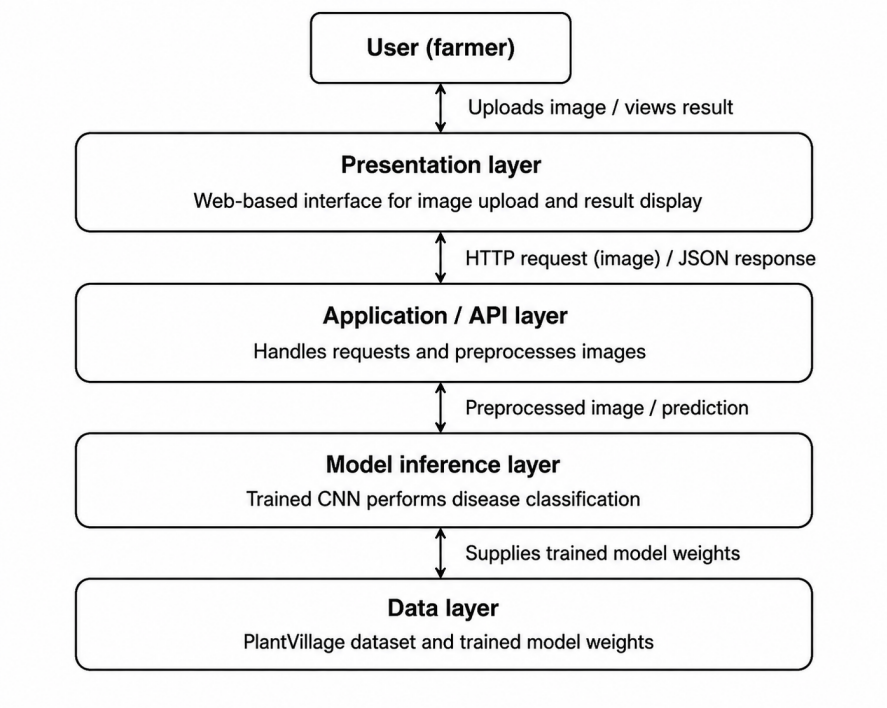


fig. 3.2: *System Architecture*

3.3 CNN Model Architecture

The CNN model is built using TensorFlow/Keras and follows a sequential architecture with multiple feature extraction and classification stages. The input is a 224x224 RGB image. The network begins with three convolutional blocks, each consisting of a Conv2D layer (with ReLU activation) followed by a MaxPooling2D layer. The number of filters doubles at each stage: 32, 64, then 128. After the convolutional blocks, the feature maps are flattened and passed through a Dense layer of 256 neurons before the final Softmax output layer with 38 neurons (one per disease class). Transfer learning is explored using MobileNetV2 pretrained on ImageNet. The base model is frozen initially to preserve learned features, then fine-tuned in a second training phase. This approach reduces training time and improves performance on the relatively small PlantVillage dataset.

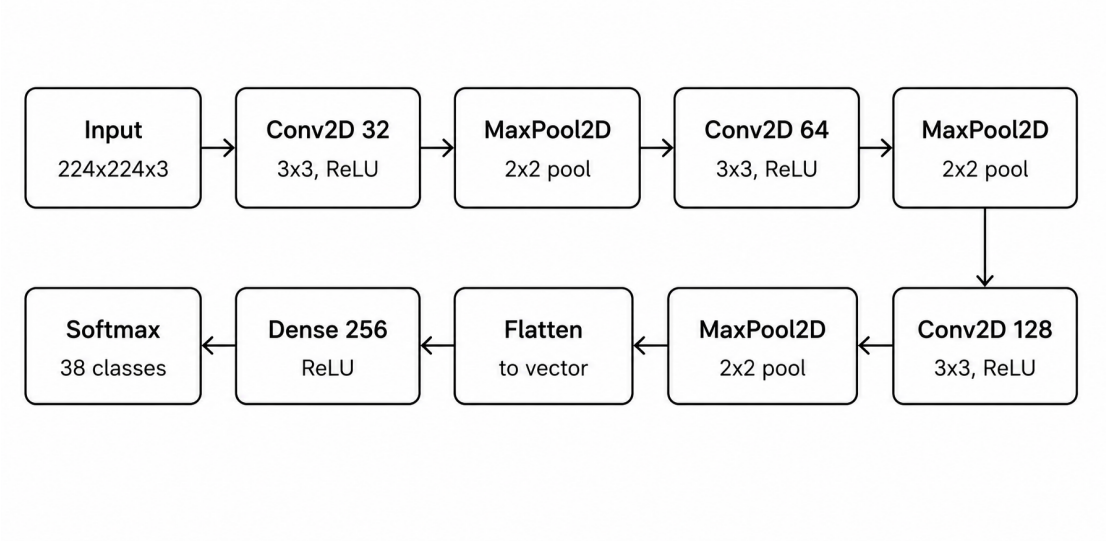


fig. 3.3: *CNN Model Architecture*

3.4 System Design

The UI module is designed as a “Herbarium specimen sheet” to provide a botanically themed, professional user experience. It features a drag-and-drop file upload zone. The backend logic handles image validation, including size and type constraints, and utilizes OpenCV to resize and normalize the image tensor to $224 \times 224 \times 3$ before passing it to the TensorFlow model.

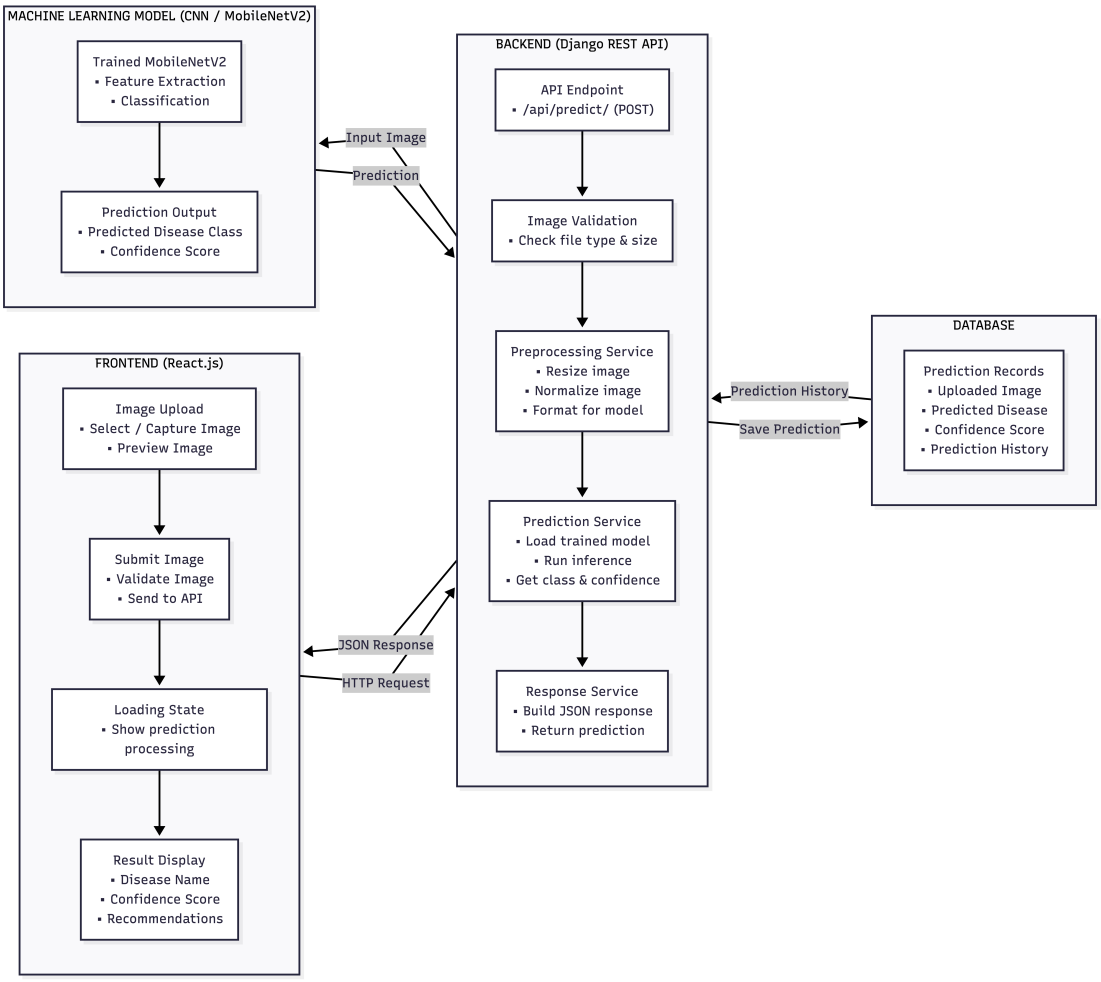


fig. 3.4: *System Design*

3.5 Technology Stack

The technology stack used in the project includes:

- **Machine Learning:** TensorFlow 2.16, Keras, Scikit-learn, OpenCV, Pandas, NumPy.
- **Backend:** Python, Django 4.2, Django REST Framework.
- **Frontend:** React.js 18, Axios, custom CSS for UI styling.
- **Server/Deployment:** Gunicorn for WSGI HTTP serving.

3.6 Data Management

The project utilizes the PlantVillage dataset containing 54,305 images and 38 classes. Data is processed locally during training. Images are augmented using Keras ImageDataGenerator with rotation, shifts, shear, and zoom to prevent overfitting.

During runtime, uploaded images are processed entirely in memory by the Django backend and are not permanently stored, ensuring user privacy and low storage overhead.

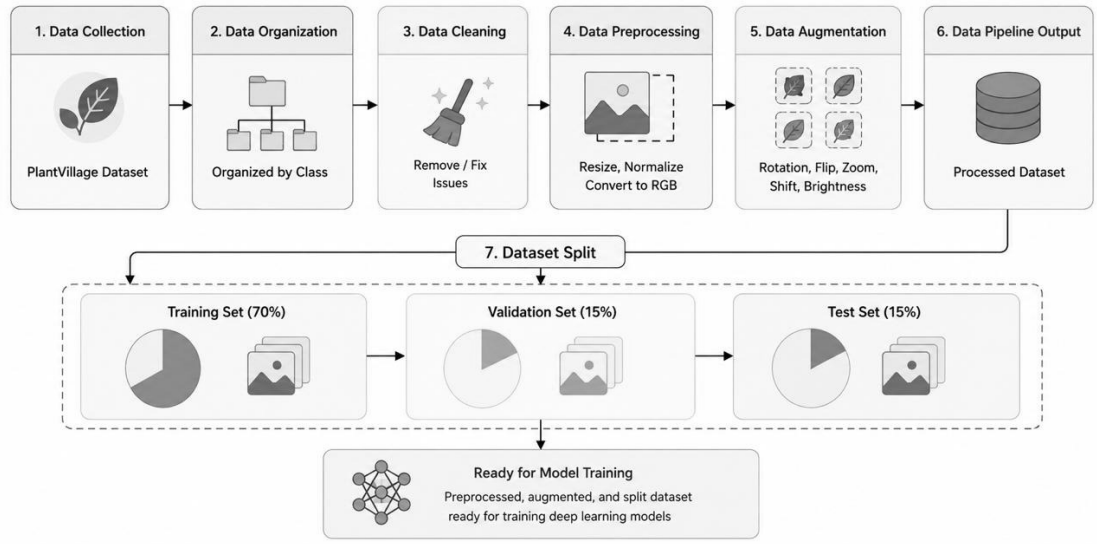


fig. 3.5: *Data Pipeline*

3.7 Testing and Validation

Testing was carried out at three levels to verify correctness of individual components and the integrated system.

Unit Testing: Image preprocessing functions (resize, RGB conversion, normalization) were verified to produce correct output shape (224×224×3) and pixel range ([0,1]). Model inference was tested against known leaf images to confirm accurate class-index mapping. The Django serializer was tested with valid and invalid file uploads to ensure proper validation.

Integration Testing: The /api/predict/ endpoint was tested via manual HTTP requests to confirm correct end-to-end flow image upload, preprocessing, inference, and JSON response. Frontend-backend integration was verified by uploading real images through the React UI and checking the displayed results against raw API responses. The model itself was validated on a held-out 15% test set, with a confusion matrix and classification report used to assess per-class performance.

User Testing: Team members and peers tested the application with their own leaf photos, including images outside the dataset's controlled conditions, to check real-world usability. Feedback on interface clarity and responsiveness will guide refinements in the remaining project phase.

3.8 Deployment Strategy

Currently, the system is configured for a local development environment.

The Django server runs locally using SQLite for configuration, and the React app is served via Webpack dev server.

Future deployment will involve containerization, such as Docker, and hosting on cloud platforms like AWS or Heroku.

CHAPTER 4
SYSTEM DESIGN AND IMPLEMENTATION

4.1 Requirements Specification

Functional Requirements

R1: UploadImage
Input: Leaf image (JPEG, PNG, or WEBP), max 10 MB
Output: Uploaded image ready for processing
Process: User selects or drags an image into the upload zone; system validates file type and size before accepting it

R2: PreprocessImage
Input: Raw uploaded image
Output: Normalized image tensor ($224 \times 224 \times 3$)
Process: System decodes the image, resizes it, converts BGR to RGB, and normalizes pixel values to [0, 1]

R3: ClassifyDisease
Input: Preprocessed image tensor
Output: Predicted disease class with confidence score
Process: System passes the tensor through the trained CNN model, applies softmax, and selects the highest-probability class

R4: DisplayResult
Input: Predicted class, confidence score, disease metadata
Output: Diagnosis card showing disease name, confidence bar, description, and treatment recommendation
Process: System maps the predicted class to the disease information dictionary and renders the result on the frontend

R5: ShowAlternateMatches
Input: Full softmax probability vector
Output: List of top secondary predictions with confidence percentages
Process: System sorts all class probabilities and displays the next highest matches below the primary result

R6: ResetUpload
Input: User action (e.g., “Choose a different photo”)
Output: Cleared upload state, ready for new image
Process: System discards the current image and result, returning the interface to its initial upload state

Non-Functional Requirements

R7: ResponseTime
Input: Submitted image
Output: Prediction result
Process: System returns a prediction within a few seconds of submission

R8: Usability
Input: N/A
Output: Consistent, intuitive interface across devices
Process: System renders a responsive layout that functions correctly on desktop and mobile browsers

R9: Privacy
Input: Uploaded image
Output: No persistent storage of user images
Process: System processes the image in memory only and discards it after returning the prediction

R10: ErrorHandling
Input: Invalid file (wrong format, oversized, corrupted)
Output: Clear error message
Process: System detects invalid input during validation and rejects it without crashing

4.2 System Overview

The completed system implements the three-tier architecture originally proposed in Section 3.2, with each layer corresponding directly to one or more of the four development iterations described in Section 3.1. This section maps the methodology to what was actually built and presents the final system structure.

Mapping Methodology to Implementation

Iteration	Methodology Focus	Implemented Component
Iteration 1	Dataset preprocessing, stratified 70/15/15 split, baseline CNN	`data_preprocessing.py`, custom 3-block CNN in `model_architecture.py`
Iteration 2	Transfer learning with MobileNetV2	MobileNetV2-based model in `model_architecture.py`, fine-tuned in `train_transfer_learning.py`
Iteration 3	Wrapping the model in a Django API	Django REST Framework backend — `/api/predict/` endpoint, `LeafImageUploadSerializer`, `model_utils.py`, `class_indices.json`
Iteration 4	Frontend development and end-to-end integration	React.js SPA — `ImageUploader` and `ResultCard` components, Axios-based API calls, CORS configuration

Table 4.1: *Mapping of development iterations to implemented system components*

Final System Structure

The completed system consists of four layers:

1. Presentation Layer (React.js SPA) — provides the "Herbarium specimen sheet" themed interface for image upload and result display, built in Iteration 4.

2. Application/API Layer (Django REST Framework) — receives uploaded images, performs validation, and orchestrates preprocessing and inference, built in Iteration 3.

3. Model Inference Layer (MobileNetV2 CNN) — performs the actual disease classification, the product of Iterations 1 and 2.

4. Data Layer — the PlantVillage dataset used during training, and the saved trained model weights (`class_indices.json` and the `.h5`/SavedModel files) loaded at inference time.

Data flows from the user's uploaded image, through preprocessing (resize, normalize, RGB conversion), into the model for inference, and back through the API as a JSON response rendered by the frontend as a diagnosis card with confidence score and treatment recommendation.

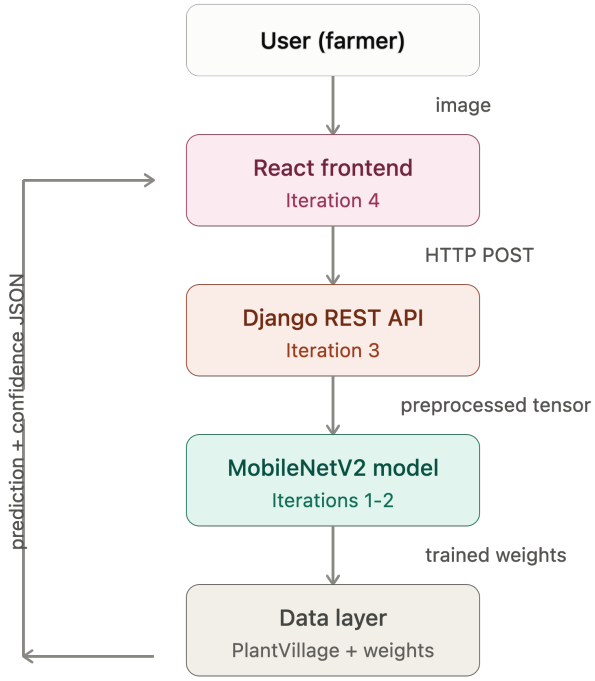


fig. 4.1: *Final as-built system architecture*

4.3 Development Environment

Category	Technology / Tool
Programming Language	Python 3.10+
Deep Learning Framework	TensorFlow 2.x / Keras
Image Processing	OpenCV 4.x
Model Architecture	Custom CNN, MobileNetV2 (Transfer Learning)
Backend Framework	Django 4.x
Frontend Framework	React.js 18 (with Axios)
Dataset	PlantVillage Dataset (54,305 images)
Development Environment	Jupyter Notebook / VS Code
Version Control	Git & GitHub
Deployment (Local)	Django Dev Server + React Dev Server

Table 4.2: *Tools and Technologies Used*

4.4 System Implementation

4.4.1 Data/Input Handling

User input enters the system when a crop leaf image is selected through the React frontend. The image is validated on the client side before submission, and revalidated independently on the server side, ensuring no invalid data reaches the model regardless of how the request was made.

Frontend capture and validation (React.js): The `ImageUploader` component's handleFiles function is invoked whenever a file is selected, whether through drag-and-drop or the file picker. It checks the file's MIME type against an accepted list and enforces a 10MB size limit before the file is allowed to proceed to preview and submission, as shown in Figure 4.2.

If either check fails, a local error message is set and displayed to the user immediately, without any network request being made — giving instant feedback and avoiding unnecessary load on the backend.

Backend capture and validation (Django REST Framework) Once a valid file is submitted, it is sent to the `/api/predict/` endpoint, where the LeafImageUploadSerializer independently re-validates the file before it can proceed to preprocessing. This serializer checks the same two conditions — content type and file size — using a defined allow-list (`ALLOWED_CONTENT_TYPES`) and a size ceiling (`MAX_UPLOAD_SIZE_MB`), as shown in Figure 4.3.

```jsx
export default function ImageUploader({ onFileSelected, previewUrl, loading, onSubmit, onClear }) {
  const [isDragOver, setIsDragOver] = useState(false);
  const [localError, setLocalError] = useState("");
  const inputRef = useRef(null);

  const handleFiles = useCallback(
    (fileList) => {
      const file = fileList && fileList[0];
      if (!file) return;
      if (!ACCEPTED_TYPES.includes(file.type)) {
        setLocalError("Please upload a JPEG, PNG, or WEBP image.");
        return;
      }
      if (file.size > 10 * 1024 * 1024) {
        setLocalError("Image is larger than 10MB. Please choose a smaller file.");
        return;
      }
      setLocalError("");
      onFileSelected(file);
    },
    [onFileSelected]
  );
```

fig. 4.2: *Client-side file validation in ImageUploader.jsx*

This server-side check is essential because client-side validation alone can be bypassed

Data flow: The complete input-handling flow, from file selection to either rejection or forwarding into the preprocessing stage, is summarized below:

1. User selects or drops a file in the browser.
2. `handleFiles()` performs a client-side check (file type and size $\leq$ 10MB).
    - If invalid: a local error message is displayed and no request is sent.
    - If valid: `onFileSelected(file)` is called, a preview is shown, and the user may submit.
3. On submission, an HTTP POST request is sent to `/api/predict/`.
4. `LeafImageUploadSerializer.validate_image()` performs a server-side check (content type and size $\leq$ `MAX_UPLOAD_SIZE_MB`).

```python
from rest_framework import serializers

MAX_UPLOAD_SIZE_MB = 10
ALLOWED_CONTENT_TYPES = ("image/jpeg", "image/jpg", "image/png", "image/webp")


class LeafImageUploadSerializer(serializers.Serializer):
    image = serializers.ImageField()

    def validate_image(self, value):
        if value.content_type not in ALLOWED_CONTENT_TYPES:
            raise serializers.ValidationError(
                f"Unsupported file type '{value.content_type}'. "
                "Please upload a JPEG, PNG, or WEBP image."
            )
        if value.size > MAX_UPLOAD_SIZE_MB * 1024 * 1024:
            raise serializers.ValidationError(
                f"Image too large. Maximum size is {MAX_UPLOAD_SIZE_MB} MB."
            )
        return value
```

fig. 4.3: *Server-side file validation in LeafImageUploadSerializer*

- If invalid: a `ValidationError` is raised and a 400 response is returned.
- If valid: the image is forwarded to the preprocessing stage

This two-layer validation — once in the browser for responsiveness, and again on the server for security — ensures that only well-formed image data (JPEG, PNG, or WEBP, under 10MB) is ever passed forward to the OpenCV preprocessing stage.

4.4.2 Pre-processing Steps

In `model_utils.py`, the uploaded byte stream is decoded via OpenCV, resized to 224×224, converted from BGR to RGB, normalized to the $[0, 1]$ range, and expanded to include a batch dimension.

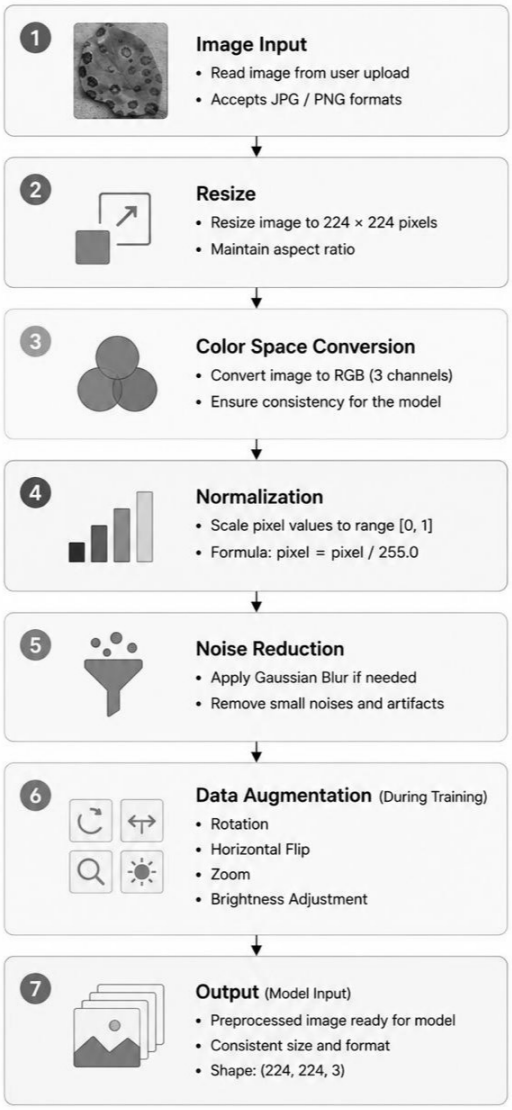


fig. 4.4: *Pre-processing Steps*

4.4.2.1 Algorithm/Logic Implementation

The core prediction logic is implemented in `predict_disease()` in `model_utils.py`, returning the top prediction and top-k probabilities per the JSON contract

Logic flow: Retrieve the lazily-loaded model via `get_model()` $\rightarrow$ preprocess image bytes $\rightarrow$ run `model.predict()` to get a 38-class softmax vector $\rightarrow$ sort indices by probability with `np.argsort` $\rightarrow$ map top-k indices to class names $\rightarrow$ return the best prediction plus top-k matches, as shown in Figure 4.5.

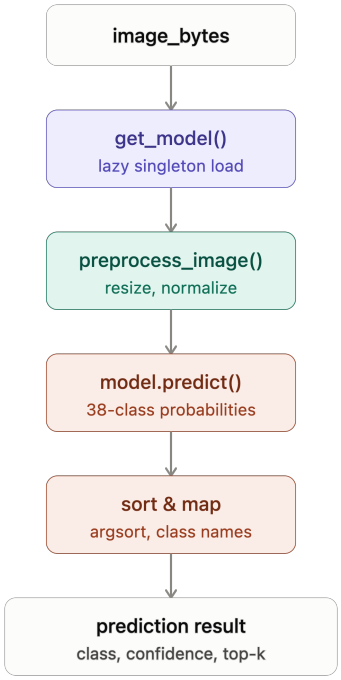


fig. 4.5: *Flow of the predict_disease function*

```python
def predict_disease(image_bytes: bytes, top_k: int = 3) -> dict:
    """
    Runs the full inference pipeline and returns the top prediction plus
    top_k class probabilities, for the JSON response described in 3.2.3.
    """
    model = get_model()
    x = preprocess_image(image_bytes)
    probs = model.predict(x, verbose=0)[0]

    top_indices = np.argsort(probs)[::-1][:top_k]
    top_predictions = [
        {
            "class_name": _idx_to_class.get(int(i), f"class_{i}"),
            "confidence": float(probs[i]),
        }
        for i in top_indices
    ]

    best = top_predictions[0]
    return {
        "predicted_class": best["class_name"],
        "confidence": best["confidence"],
        "top_predictions": top_predictions,
    }
```

fig. 4.6: *Implementation of predict_disease() in model_utils.py*

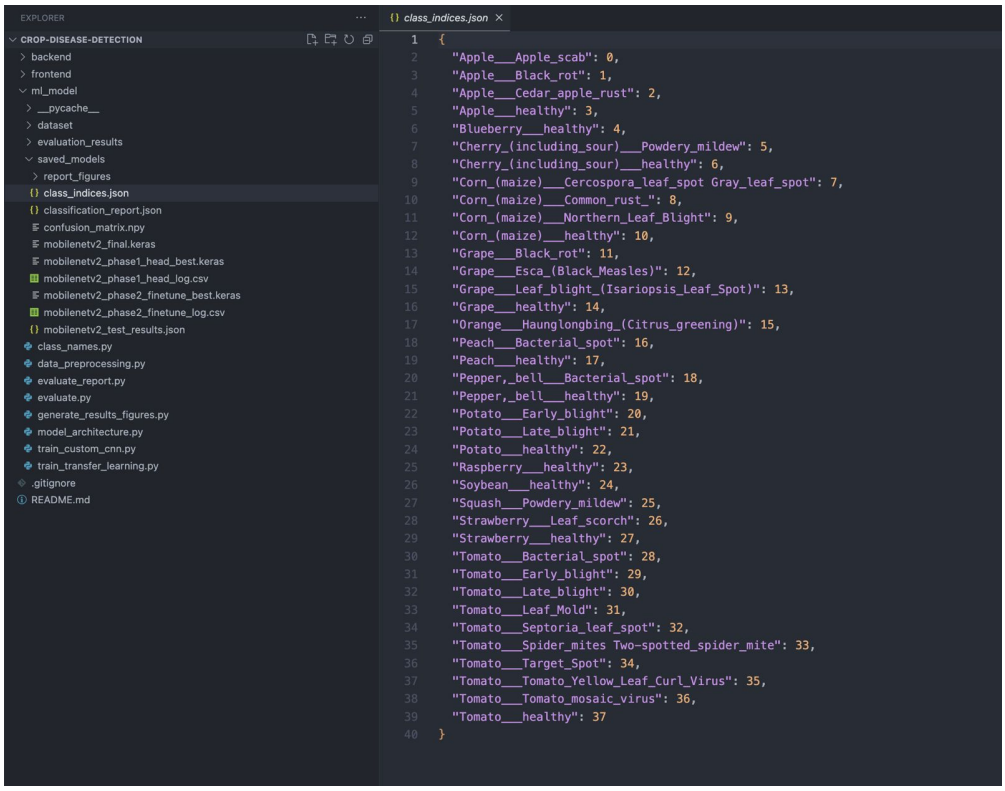


fig. 4.7: *Disease class*

4.4.3 Module Design & Integration

The system was built as three independent modules — ML Pipeline, Backend API, and Frontend Application — each communicating through a fixed interface so changes in one did not require changes in the others.

Module boundaries:

- ML Pipeline — training scripts, model architecture, and outputs a saved model plus `class_indices.json`.
- Backend API — loads the saved model, validates/preprocesses images, runs inference, returns JSON via `/api/predict/`.
- Frontend — consumes the REST API only; has no knowledge of the model or Python internals.

Integration approach: Modules were integrated as soon as each was ready, not all at once:

1. Model and `class_indices.json` loaded into the backend via a lazy singleton (avoids slow startup).
2. Frontend built against the stable `/api/predict/` JSON response, using Axios and CORS.
3. Full flow (upload $\rightarrow$ validate $\rightarrow$ preprocess $\rightarrow$ predict $\rightarrow$ display) tested end-to-end.

4.4.4 Interface and Visualisation

The frontend provides a stylized, intuitive interface. A drag-and-drop zone with animated scan-lines during processing enhances the user experience.

Results are displayed in a card featuring a “healthy” or “diseased” stamp, a visual confidence bar, and a list of secondary predictions if applicable.

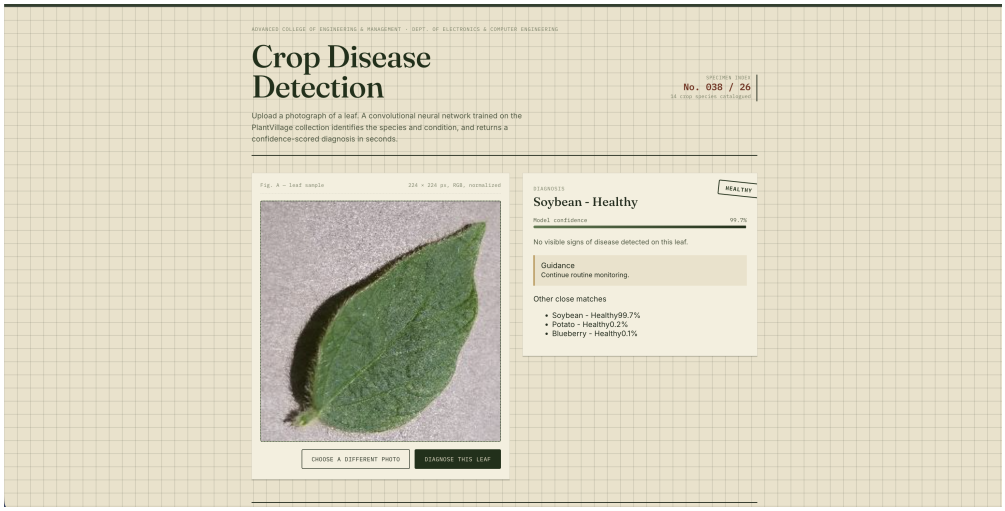


fig. 4.8: *User interface*

4.4.5 Deployment and Execution

The system currently runs in a local development environment, with the Django backend and React frontend served independently and communicating over HTTP during development. Cloud deployment (Docker, AWS/Heroku) is planned as future work but not yet implemented.

Prerequisites:

- Python 3.10+
- Node.js and npm
- Git

Backend Setup:

[language=bash] cd crop-disease-detection/backend python -m venv venv source venv/bin/activate pip install -r requirements.txt python manage.py migrate python manage.py runserver

This starts the Django REST API on `http://localhost:8000`, using SQLite as the local database for configuration/session data. The trained model and `class_indices.json` are loaded lazily (on the first prediction request) from `ml_model/saved_models/`, rather than at server startup, to reduce initial load time.

Frontend Setup:

[language=bash] cd crop-disease-detection/frontend npm install npm start

This starts the React development server on `http://localhost:3000`, which proxies API requests to the Django backend at `http://localhost:8000/api/predict/`.

Configuration:

- CORS: configured in the backend's `settings.py` to explicitly allow requests from `http://localhost:3000`, enabling cross-origin communication between the two dev servers.
- Environment variables: model path and any API keys/secrets are read from environment variables rather than hardcoded, allowing the same codebase to be reconfigured for different environments.

Future Deployment Plan:

The current local setup is intended as a development configuration only. Planned production deployment includes:

- Containerization of both backend and frontend using Docker
- Cloud hosting on a platform such as AWS or Heroku
- Replacing SQLite with a production-grade database if persistent storage needs arise
- Serving the React build as static files via a production web server (e.g., Nginx) rather than the development server

4.4.6 Challenges and Solutions

Class index mismatch: Keras's label ordering during training did not always match inference assumptions, causing wrong disease names to be returned. This was solved by exporting the mapping to `class_indices.json` during training and loading it dynamically at inference.

Slow model loading: Loading the TensorFlow model on every request was too slow; loading at startup delayed server availability. This was solved with a lazy singleton pattern in `model_utils.py` — the model loads once, on the first request, and is reused afterward.

CORS blocking frontend-backend calls: The React dev server (port 3000) and Django API (port 8000) run separately, so the browser blocked cross-origin requests. This was solved by configuring CORS in `settings.py` to allow the frontend's origin.

Unreliable client-only validation: File type/size checks in the React uploader could be bypassed by calling the API directly. This was solved by duplicating the same validation server-side in `LeafImageUploadSerializer`.

Accuracy vs. computational cost (partially resolved): Training a custom CNN from scratch was slow and less accurate. Transfer learning with MobileNetV2 improved accuracy to $\sim$97% while cutting training time, but this is a trade-off, not a full fix — the model is still trained only on lab-condition PlantVillage images, and its accuracy on real-world field photos has not been formally tested, which remains a limitation.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Results

The project has successfully reached a fully integrated, functional state. The machine learning pipeline successfully processes the PlantVillage dataset. Both the custom CNN and MobileNetV2 architectures compile and train successfully. The Django REST API successfully validates uploads, handles OpenCV preprocessing, and executes model inference without crashing. The React SPA successfully communicates with the backend, displaying dynamic result cards with accurate disease information.

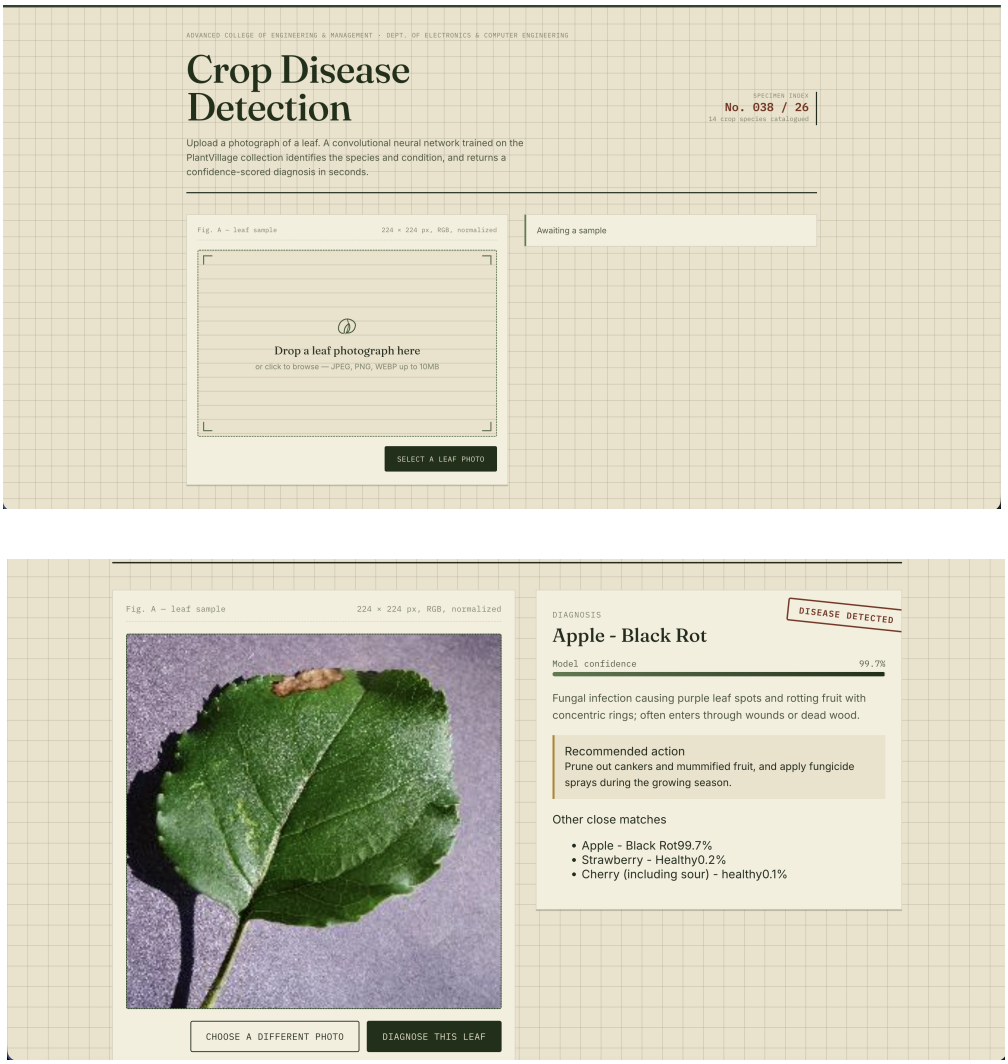


fig. 5.1: *User interface*

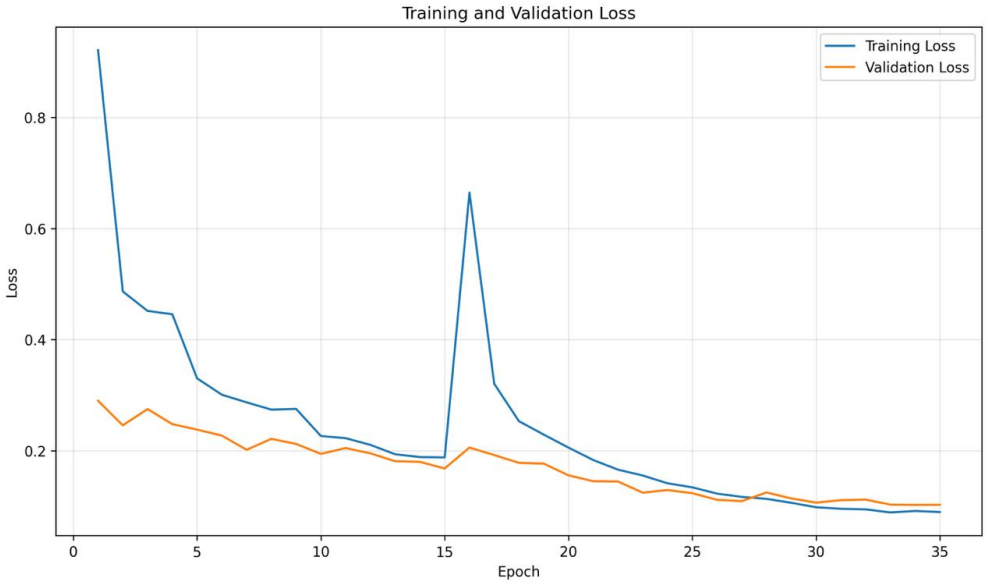


fig. 5.2: *Training and Validation Loss*

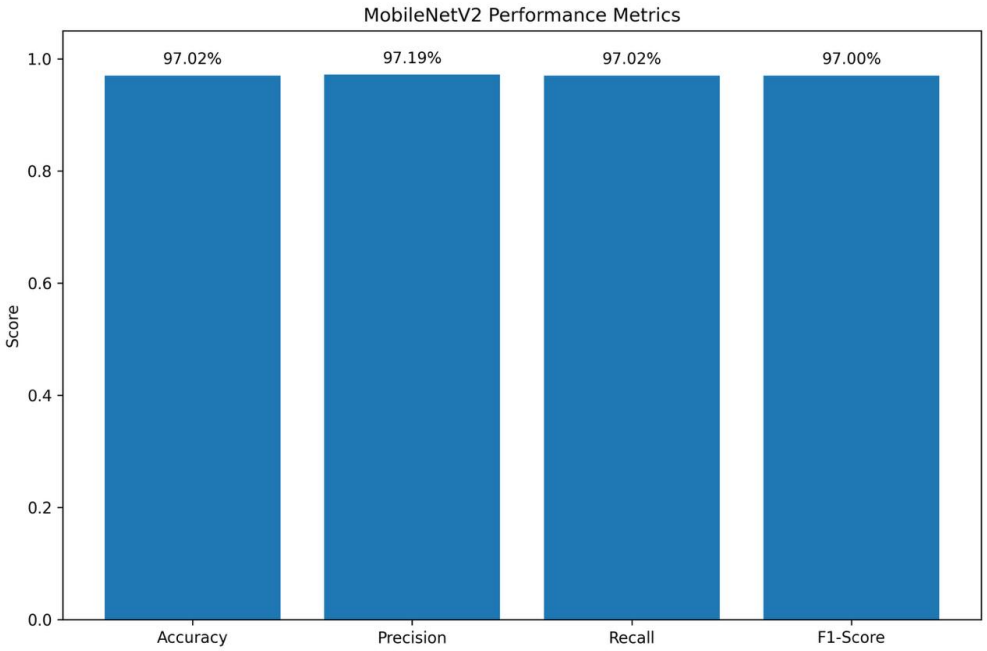


fig. 5.3: *Performance Matrix*

5.2 Discussion

Compared to existing academic notebooks, this project provides a tangible, usable product. Unlike commercial applications that hide implementation details, this system offers a transparent, open-source pipeline from data splitting to UI presentation.

The inclusion of the “Herbarium” UI and detailed agricultural recommendations sets it apart from simple classification demos.

5.3 Comparative Analysis

Compared to existing academic notebooks, this project provides a tangible, usable product.Unlike commercial applications that hide implementation details, this system offers a transparent, open-source pipeline from data splitting to UI presentation. The inclusion of the “Herbarium” UI and detailed agricultural recommendations sets it apart from simple classification demos.

CHAPTER 6
CONCLUSIONS & RECOMMENDATIONS

6.1 Conclusions

This project addressed farmers' inability to quickly and accurately diagnose crop diseases without expert access, particularly relevant in Nepal where manual inspection is slow and often inaccessible. The solution combined a CNN-based classifier with a full web delivery system — Django REST API backend and React.js frontend — built iteratively: baseline CNN → MobileNetV2 transfer learning → API integration → frontend integration.

The completed system achieved approximately 97% accuracy, precision, recall, and F1-score on the PlantVillage test set, and works end-to-end: users upload a leaf image and receive a diagnosis with confidence score and treatment recommendations through a herbarium-themed interface.

Key contributions of the project include the following:

- Demonstrated that MobileNetV2 transfer learning outperforms a custom CNN with less training time.
- Delivered a complete, deployable three-tier system rather than an isolated notebook model.
- Solved practical serving challenges, including class-index consistency, lazy model loading, and in-memory privacy-conscious processing.

Objective	**Status**
Dataset collection & preprocessing	Achieved
CNN design and training	Achieved
Evaluation via accuracy, precision, recall, F1-score	Achieved (approximately 97%)
Django REST API	Achieved
React.js web application	Achieved
Accessible tool for farmers	Partially achieved — functional prototype, but untested on real field images and not yet deployed beyond local development

Table 6.1: *Evaluation of project objectives against outcomes*

6.2 Future Enhancements

- Field validation: test on real farmer-captured images (variable lighting and backgrounds) to measure real-world accuracy drop.
- Cloud deployment: containerize with Docker and deploy on AWS or Heroku for actual accessibility.
- Offline support: convert to TensorFlow Lite for low-connectivity rural use.
- Localization: add Nepali-language support to the interface.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[2] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[3] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, 2015.

[4] P. Tm, A. Pranathi, K. SaiAshritha, N. B. Chittaragi, and S. G. Koolagudi, “Tomato leaf disease detection using convolutional neural networks,” in *Proceedings of the 11th International Conference on Contemporary Computing (IC3)*, (Noida, India), 2018.

[5] F. Chollet, *Deep Learning with Python*. Manning Publications, 2nd ed., 2021.